

Progetto per il Corso di Sistemi Intelligenti per Internet

Per la realizzazione di questo **Progetto** ho addestrato e analizzato le caratteristiche di alcuni Modelli per il **Rilevamento delle Fake News**.

In particolare i Modelli presi in considerazione possono essere divisi in 2-Gruppi:

- **Modelli** basati sul *Machine Learning*
- **Modello** basato sul *Deep Learning*

Il **DataSet** utilizzato per l'**Addestramento** e la **Valutazione** e' lo stesso per ogni **Modello**:

"<https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset?resource=download>"

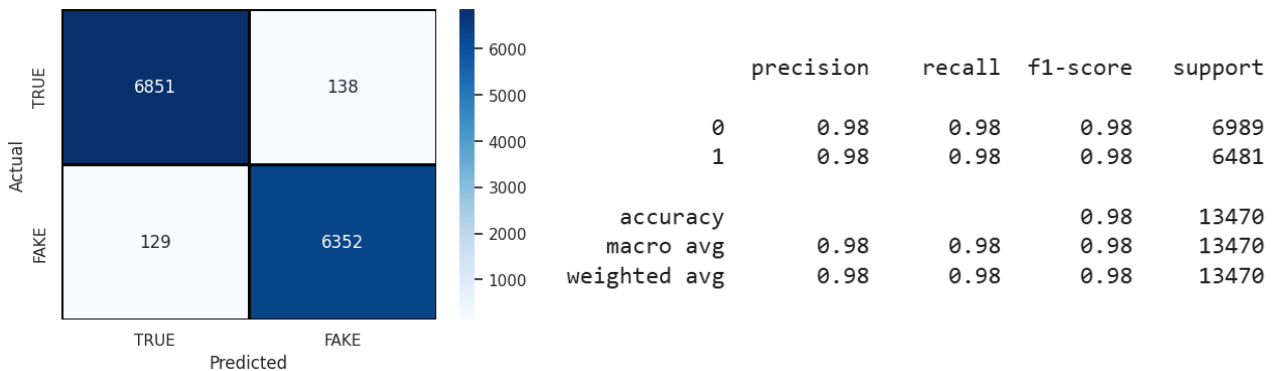
Il **Lavoro** svolto su ogni **Modello** può essere diviso in:

1. *Preparazione dei Dati (- Preprocessing & Word Optimization)*
2. *Estrazione delle Feature (- Vettorizzazione TF-IDF)*
3. *Suddivisione del Dataset (- Training & Test Set)*
4. *Addestramento del Modello*
5. *Analisi dei Risultati*

Modelli basati sul Machine Learning:

1. *Regressione Lineare*

Questo Modello ha prodotto dei risultati **Discreti** nonostante la sua semplicità.

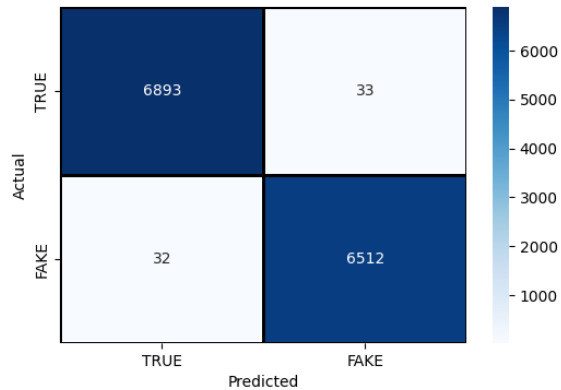


Le ragioni possono essere svariate, in particolare:

- Un modello basato su **Linear Regression** può funzionare meglio di un **Decision Tree** sugli stessi Dati principalmente a causa della **linearità intrinseca dei pattern testuali (- TF-IDF)** e del rischio di **Overfitting** dell'**Albero** su **Dataset** ad **Alta Dimensionalità**
- I **Modelli Lineari** hanno strumenti matematici molto potenti per difendersi dal **Rumore**, come la **Regolarizzazione L1 (Lasso)** o **L2 (Ridge)**.
(- Ad es. Se nel **Dataset** ci sono parole rare o errori di battitura che compaiono per puro caso solo nelle **Fake News**, la **Regolarizzazione** azzerava o riduce drasticamente il loro **Peso**)

2. Albero di Decisione

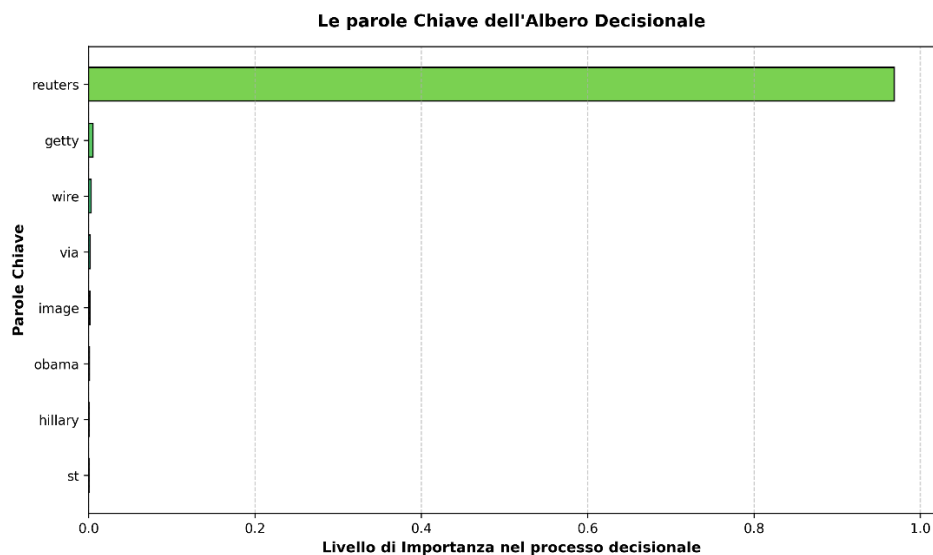
Nel caso del Modello basato su Decision Tree, inizialmente sembrava essere quello più performante. Otteneva risultati che superavano di gran lunga i Modelli basati su **Linear Regression** e **Random Forest**, raggiungendo un **Accuratezza** anche pari ad **1.0**



Tali **Risultati** erano talmente perfetti da necessitare un **Ulteriore Analisi** per scoprire i **Motivi** della loro presenza.

In particolare, mi sono concentrato nello scoprire quale fosse la **Struttura dell'Albero Decisionale** dopo l'**Addestramento**, ma più nel dettaglio, quali fossero le **Parole Chiave** che inficiavano maggiormente nel **Potere Decisionale** del **Modello**.

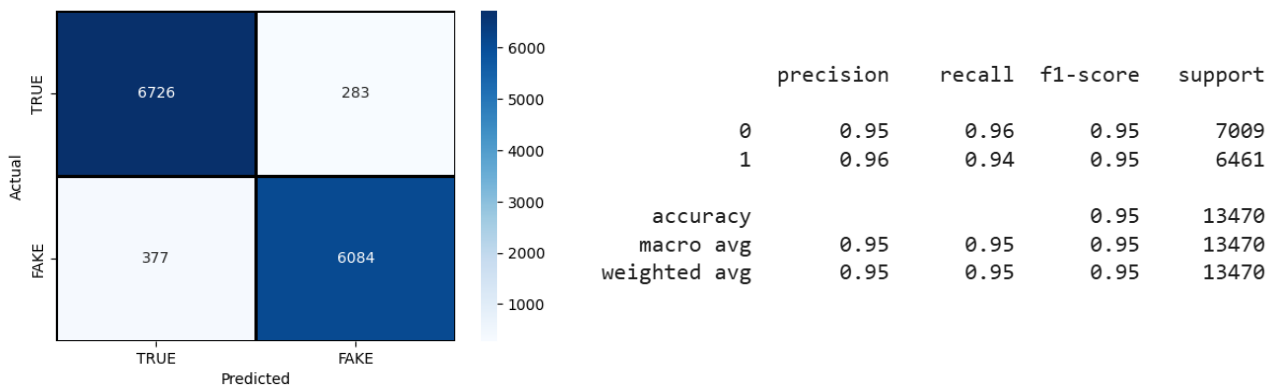
Effettuando il **Plot** di questa **Analisi**, si capisce immediatamente come i **Motivi** dietro l'efficienza del **Modello**, non derivino da pregi legati al **Modello** stesso, ma piuttosto da **Problemi** ereditati dal **Dataset**.



La **Parola "Reuters"** risulta essere quella con **Maggiore Potere Decisionale** all'interno dell'**Albero**. Siccome **"Reuters"** è il nome di un **Testata Giornalistica**, si può immaginare molto facilmente dove risieda il **Problema** dei risultati iniziali.

Tale **Parola** deve essere riportata all'interno delle **Notizie** che appartengono solamente al gruppo delle **Fake News** (- o delle **True News**). In questo modo, l'**Albero**, quando vede questa **Parola** riportata nel **Testo**, sa immediatamente che tipo di **Classificazione** compiere.

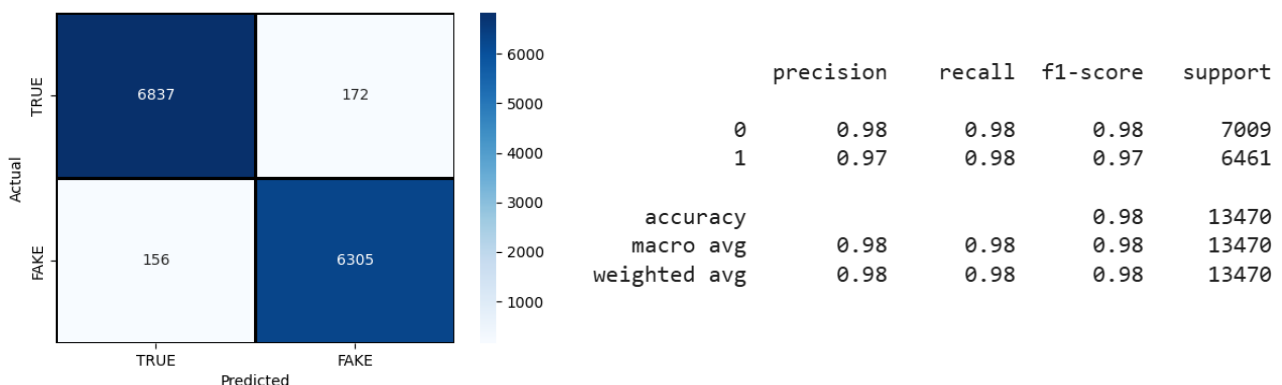
Rimuovendo questa **Parola** dal **Dataset** (- attraverso un opportuna **Ottimizzazione del Testo**), si può notare un **notevole cambiamento** nei **Risultati del Modello**, il quale adesso **Performa Peggio** di quello basato su **Linear Regression**.



3. Random Forest

Il Modello basato su **Random Forest** è sicuramente quello più complesso tra tutti i **Modelli di Machine Learning** presi in considerazione.

Di conseguenza ci si aspetterebbe che sia quello con i **Risultati Migliori**. Tuttavia, inizialmente non è stato così, infatti presentava dei **Risultati** molto **Simili** o addirittura **Peggiori** rispetto al **Modello** basato su **Linear Regression**.



Attraverso un ulteriore **Ragionamento**, ho pensato che tali **Risultati** potessero dipendere da un **Overfitting** della **Foresta di Alberi Decisionali**.

Per cercare di evitare questo **Overfitting** ho quindi deciso di **Aggiungere degli Iperparametri** al Modello e di **Modificare** leggermente la **Vettorizzazione delle Parole**:

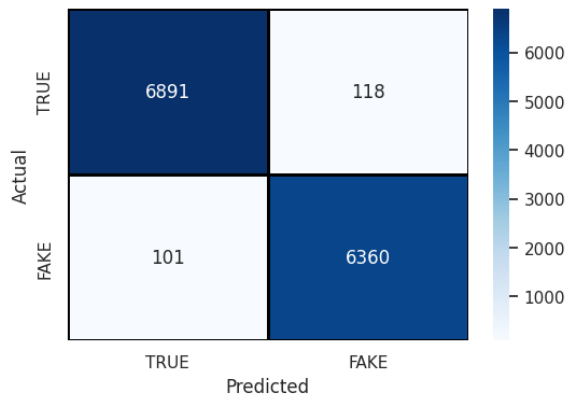
- Vettorizzazione

- `max_features=2000` (- Limitare il numero di Parole del Vocabolario)
- `min_df=5` (- Quando viene costruito il Vocabolario, bisogna ignorare le Parole che hanno una Frequenza più bassa del valore che viene passato)

- Iperparemetri

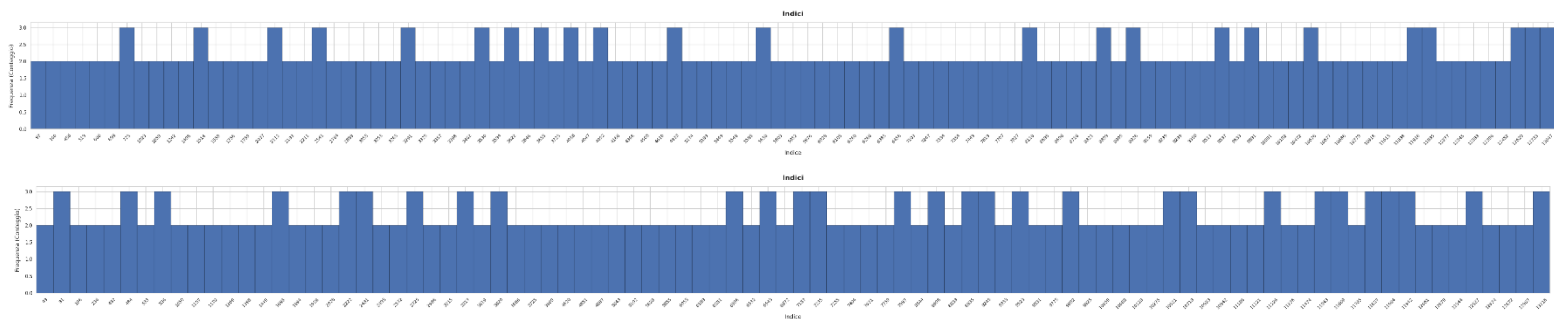
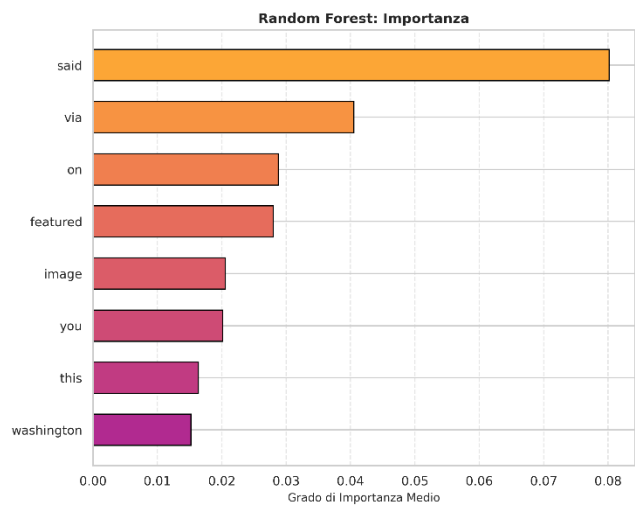
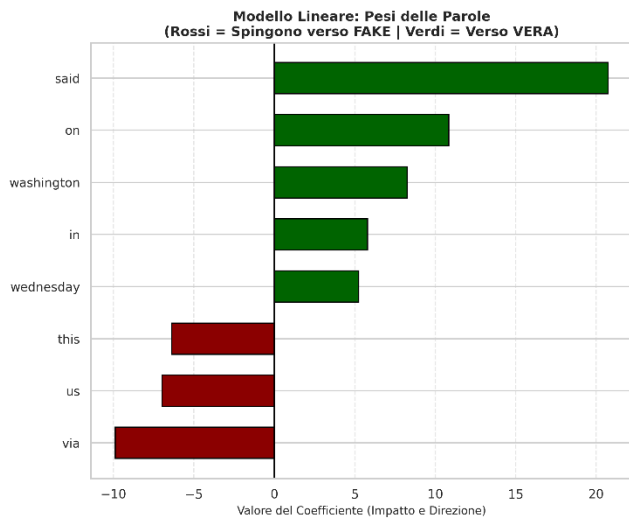
- `max_depth=30` (- Bloccare la Profondità)
- `n_estimators=300` (- Aggiungere Alberi alla Foresta)

Grazie a questi cambiamenti, riusciamo a **Eliminare l'Overfitting** e ottenere dei **Risultati Migliori** rispetto quelli dei **Modelli** visti in precedenza.



	precision	recall	f1-score	support
0	0.99	0.98	0.98	7009
1	0.98	0.98	0.98	6461
accuracy			0.98	13470
macro avg	0.98	0.98	0.98	13470
weighted avg	0.98	0.98	0.98	13470

Confronto: Modello Lineare vs Random Forest



(- Istogramma che rappresenta gli indici dei Testi nel Dataset che sono stati rilevati come Falsi Positivi o Falsi Negativi da più di un Modello)

Modello basato su Deep Learning:

Per evitare i problemi affrontati in precedenza con il **Dataset**, il **Modello LSTM** è stato addestrato su dei **Dati** che hanno affrontato un **Preprocessing** e **WordOptimizer**, leggermente diversi.

In questo caso di studio ho usato dei **Vettori Globali** per la rappresentazione delle **Parole** (- **Global Vectors for Word Representation "GLOVE"**). **GLOVE** è un tipo di **Apprendimento Non Supervisionato** per ottenere la **Rappresentazione Vettoriale** delle **Parole**.

L'idea di base dietro l'**Embedding** è che può essere utile a stabilire la **Rappresentazione Contestuale** delle **Parole** in un **Documento**, la loro **Semantica** e le **Relazioni** che condividono. Funziona bene con **LSTM** in quanto il **Modello** diviene **più capace nel comprendere il Testo** in termini di conoscenza contestuale, rispetto ai metodi che sfruttano la sola **Vettorizzazione delle Parole**.

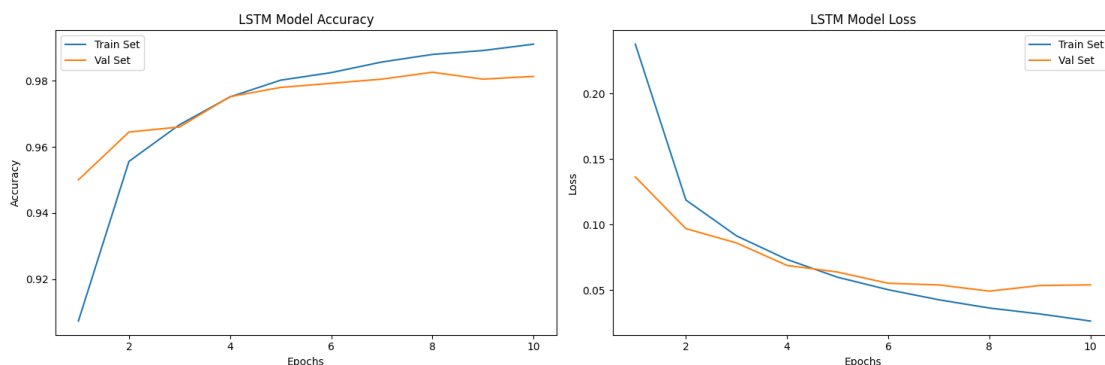
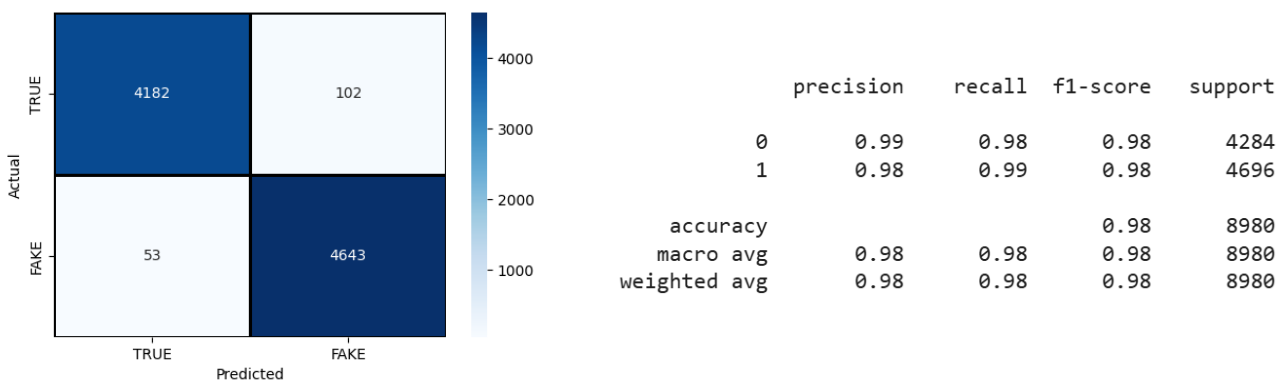
Per l'addestramento della **Rete Neurale** inizialmente sono stati scelti degli **Iperparametri** precisi.

```
# Configuration Settings
MAX_SEQUENCE_LENGTH = 100
MAX_VOCAB_SIZE = 20000
EMBEDDING_DIM = 50
VALIDATION_SPLIT = 0.2
BATCH_SIZE = 32
EPOCHS = 10
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 100)	0
embedding (Embedding)	(None, 100, 50)	1,000,000
bidirectional (Bidirectional)	(None, 100, 30)	7,920
global_max_pooling1d (GlobalMaxPooling1D)	(None, 30)	0
dense (Dense)	(None, 1)	31

Total params: 1,007,951 (3.85 MB)
Trainable params: 7,951 (31.06 KB)
Non-trainable params: 1,000,000 (3.81 MB)

I **Risultati** ottenuti alla fine dell'addestramento coincidono con quelli che ci si poteva aspettare. Infatti, la **Rete** ha **performato meglio** rispetto a tutti i **Modelli** basati su **Tecniche di Machine Learning**.



Tuttavia, per riuscire a spremere al massimo le **Performance** di questo **Modello**, ho intrapreso un ultimo passo implementando una procedura di **Ottimizzazione degli Iperparametri**.

Per svolgere questo compito ho sfruttato la **Libreria "keras_tuner"**.

Il **Modello della Rete** è rimasto identico a prima, ma ho inserito diverse possibilità nella variazione possibile degli **Iperparametri**:

- **Unità LSTM** tra '8' e '64'
- **Ottimizzatore** tra 'adam' e 'rmsprop'
- **Livello di Embedding** è 'Trainable' True o False
- **Learning Rate** tra '1e-4' e '1e-2'

Il **Tuner** usa **RandomSearch** per effettuare l'**Ottimizzazione** (- **Efficienza e Semplicità**)

Alla fine dell'**Ottimizzazione**, otteniamo questi **Risultati**:

```
Trial 10 Complete [00h 10m 38s]
val_accuracy: 0.9912163019180298
```

```
Best val_accuracy So Far: 0.9912163019180298
Total elapsed time: 01h 19m 34s
```

Il tuning è completato. Il numero ottimale di unità LSTM è 48,
l'ottimizzatore migliore è rmsprop con un learning rate di 0.0044908815472004615.
Embedding impostato su Trainable = True.

Ovviamente i **Risultati** che si ottengono dopo aver **Addestrato la Rete** con l'utilizzo di questi **Iperparametri**, sono **di gran lunga superiori** a tutti quelli visti fino ad ora.

